LeetCode in C

Study guide: how it works, patterns, C templates, and a roadmap of problems

1. How LeetCode works in C

You never write `main`. LeetCode gives you a function signature and calls it with many test cases. You write only that function (plus helpers). Headers like stdio, stdlib, string, stdbool, and limits are already included.

```
// Problem: Two Sum. LeetCode gives you this:
int* twoSum(int* nums, int numsSize, int target, int* returnSize);
```

Part of the signature	Meaning
`int* nums, int numsSize`	An array does not carry its length in C, so the size is passed separately.
`int* returnSize`	An **output parameter**: you must write the length of your returned array into `*returnSize`.
Return type `int*`	Return a **malloc'd** array. LeetCode frees it after checking.
`int** returnColumnSizes`	For 2D results: allocate an int array and store each row's length in it.
`char** grid, int gridSize, int* gridColSize`	2D input: gridSize = rows, gridColSize[0] = columns.
`struct ListNode*`	Linked list nodes are already defined. Do not redefine them.
`struct TreeNode*`	Tree nodes are already defined (val, left, right).

Test your solution locally first

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

/* paste your solution here */

int main(void)
{
    int nums[] = {2, 7, 11, 15};
    int size;
    int *r = twoSum(nums, 4, 9, &size);
    printf("%d %d (size %d)\n", r[0], r[1], size);
    free(r);
    return 0;
}
```

Compile with `gcc -Wall -Wextra -g -fsanitize=address,undefined file.c`. This catches most bugs before LeetCode shows you a vague runtime error.

2. Big-O: the cheat sheet

Complexity	Name	Example
O(1)	Constant	Array index, hash lookup, push on a stack
O(log n)	Logarithmic	Binary search, heap push/pop
O(n)	Linear	One loop, linked-list traversal
O(n log n)	Linearithmic	Sorting (qsort), merge sort
O(n^2)	Quadratic	Two nested loops, bubble sort
O(2^n)	Exponential	All subsets, naive Fibonacci
O(n!)	Factorial	All permutations

Use the constraints to pick the algorithm

LeetCode states the input size. A typical judge handles about 10^8 simple operations per second, so the limit tells you what is allowed.

Input size n	Target complexity	Typical techniques
n <= 10	O(n!)	Permutations, brute-force backtracking
n <= 20	O(2^n)	Subsets, bitmask, backtracking
n <= 500	O(n^3)	Triple loops, Floyd-Warshall
n <= 10^4	O(n^2)	Nested loops, simple DP
n <= 10^5	O(n log n)	Sorting, heap, binary search, divide and conquer
n <= 10^6	O(n)	Hash table, two pointers, sliding window, prefix sums
n > 10^6	O(log n) or O(1)	Math, binary search on the answer

3. A 6-step method for any problem

- **1. Understand.** Restate the problem in your own words. What are the input, output, and constraints?
- **2. Examples and edge cases.** Write 2-3 examples by hand. Think: empty input, one element, duplicates, negatives, all equal, maximum size.
- **3. Brute force.** Say the simplest solution and its complexity out loud. It is your baseline and usually correct.
- **4. Optimize.** Ask what repeated work you can remove. Can a hash table, sorting, two pointers, or a stack help? Use the pattern table below.
- **5. Code.** Write clean code with clear variable names. Allocate, set returnSize, and free temporary memory.
- **6. Test.** Walk through your examples line by line, then your edge cases. Check the complexity.

Tip: If you are stuck for 20-30 minutes, read the hint or solution, then close it and re-implement from memory the next day. Learning from solutions is normal and is how patterns stick.

4. Pattern recognition: clue to technique

If you see...	Try...	Example problems
Pair or complement lookup, duplicates, counting	Hash table / counting array	1, 217, 242, 49
Sorted array, pairs, palindromes, in-place removal	Two pointers	125, 15, 11, 26, 283
Longest / shortest *contiguous* subarray or substring	Sliding window	3, 121, 424
Range sums, product of others	Prefix sums	238, 560
Sorted input, or 'minimize the max / maximize the min'	Binary search (also on the answer)	704, 33, 875
Matching brackets, next greater element, undo	Stack (monotonic stack)	20, 155, 739
Linked list: reverse, middle, cycle, merge	Pointers: prev/curr, slow/fast, dummy node	206, 141, 21, 876
Tree: depth, paths, validity	DFS recursion	104, 226, 98, 543
Tree or grid by levels, shortest path (unweighted)	BFS with a queue	102, 200

If you see...	Try...	Example problems
All combinations, subsets, permutations	Backtracking	78, 46, 39, 79
Count ways, min/max cost, choices that repeat	Dynamic programming	70, 198, 322, 300, 1143
Top K, k-th largest, merge K sorted	Heap (priority queue)	215, 347, 703
Connections, dependencies, islands	Graph DFS/BFS, topological sort	200, 207, 133
Single number, powers of two, counting bits	Bit manipulation (XOR, shifts)	136, 191, 338, 268
Overlapping intervals	Sort by start, then sweep	56

5. Patterns with C templates

Each template is a skeleton you should be able to write from memory. The LeetCode numbers show where to practice.

5.1 Hash table (C has no built-in one)

For Two Sum (#1), a small open-addressing table is enough. The size is a power of two, larger than twice the element count.

```
#define SZ 32768                        /* power of two */
typedef struct { int key, val, used; } Entry;

static unsigned hash(int k) { return ((unsigned)k * 2654435761u) & (SZ - 1); }

int* twoSum(int* nums, int n, int target, int* returnSize)
{
    Entry *t = calloc(SZ, sizeof(Entry));
    int *res = malloc(2 * sizeof(int));
    *returnSize = 2;
    for (int i = 0; i < n; i++)
    {
        int need = target - nums[i];
        for (unsigned h = hash(need); t[h].used; h = (h + 1) & (SZ - 1))
            if (t[h].key == need)
            {
                res[0] = t[h].val; res[1] = i;
                free(t);
                return res;
            }
        unsigned h = hash(nums[i]);
        while (t[h].used) h = (h + 1) & (SZ - 1);
        t[h] = (Entry){ nums[i], i, 1 };
    }
    free(t); *returnSize = 0;
    return res;
}
```

Counting arrays are even simpler when the key range is small, such as lowercase letters or ASCII:

```
int cnt[26] = {0};
for (int i = 0; s[i]; i++) cnt[s[i] - 'a']++;     /* Valid Anagram #242 */
```

5.2 Two pointers

```
/* Valid Palindrome #125 (ignore non-alphanumeric, ignore case) */
bool isPalindrome(char *s)
{
    int l = 0, r = strlen(s) - 1;
    while (l < r)
    {
        while (l < r && !isalnum((unsigned char)s[l])) l++;
        while (l < r && !isalnum((unsigned char)s[r])) r--;
        if (tolower((unsigned char)s[l]) != tolower((unsigned char)s[r])) return false;
        l++; r--;
    }
    return true;
}

/* Two pointers on a sorted array: find a pair summing to target */
int l = 0, r = n - 1;
while (l < r)
{
    int sum = a[l] + a[r];
    if (sum == target) { /* found */ break; }
    else if (sum < target) l++;
    else r--;
```

```
}
/* 3Sum #15: sort, fix a[i], then run this on i+1..n-1; skip duplicates. */
```

5.3 Sliding window

```
/* Longest Substring Without Repeating Characters #3 */
int lengthOfLongestSubstring(char *s)
{
    int last[256];
    for (int i = 0; i < 256; i++) last[i] = -1;
    int best = 0, left = 0;
    for (int r = 0; s[r]; r++)
    {
        unsigned char c = s[r];
        if (last[c] >= left) left = last[c] + 1;   /* shrink window */
        last[c] = r;
        if (r - left + 1 > best) best = r - left + 1;
    }
    return best;
}
```

General shape: expand *right* every step, shrink *left* while the window is invalid, and update the answer each time the window is valid.

5.4 Prefix sums / running values

```
/* Product of Array Except Self #238: no division, O(n) */
int* productExceptSelf(int* nums, int n, int* returnSize)
{
    int *res = malloc(n * sizeof(int));
    *returnSize = n;
    int prefix = 1;
    for (int i = 0; i < n; i++) { res[i] = prefix; prefix *= nums[i]; }
    int suffix = 1;
    for (int i = n - 1; i >= 0; i--) { res[i] *= suffix; suffix *= nums[i]; }
    return res;
}
```

```
/* Maximum Subarray #53 (Kadane) */
int maxSubArray(int* nums, int n)
{
    int cur = nums[0], best = nums[0];
    for (int i = 1; i < n; i++)
    {
        cur = (cur > 0) ? cur + nums[i] : nums[i];
        if (cur > best) best = cur;
    }
    return best;
}
```

5.5 Binary search

```
/* Classic: find target, return index or -1 (#704) */
int search(int* nums, int n, int target)
{
    int lo = 0, hi = n - 1;
    while (lo <= hi)
    {
        int mid = lo + (hi - lo) / 2;          /* avoids overflow */
        if (nums[mid] == target) return mid;
        if (nums[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

```
/* Binary search on the ANSWER: smallest x where ok(x) is true (e.g. #875) */
```

```
int lo = 1, hi = MAX_POSSIBLE;
while (lo < hi)
{
    int mid = lo + (hi - lo) / 2;
    if (ok(mid)) hi = mid;                    /* mid might be the answer */
    else lo = mid + 1;
}
return lo;
```

5.6 Sorting with qsort

```
/* Safe comparator: never use (a - b), it can overflow */
int cmp(const void *a, const void *b)
{
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);
}
qsort(nums, n, sizeof(int), cmp);

/* Intervals #56: sort an array of int[2] by start */
int cmp_iv(const void *a, const void *b)
{
    const int *x = *(const int **)a, *y = *(const int **)b;   /* int** rows */
    return (x[0] > y[0]) - (x[0] < y[0]);
}
```

5.7 Stack

```
/* Valid Parentheses #20 */
bool isValid(char *s)
{
    int n = strlen(s), top = 0;
    char *st = malloc(n + 1);
    bool ok = true;
    for (int i = 0; i < n && ok; i++)
    {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{') st[top++] = c;
        else if (top == 0) ok = false;
        else
        {
            char o = st[--top];
            if ((c == ')' && o != '(') || (c == ']' && o != '[')
                || (c == '}' && o != '{'))
                ok = false;
        }
    }
    ok = ok && top == 0;
    free(st);
    return ok;
}
```

Monotonic stack (Daily Temperatures #739): keep a stack of indices whose values are decreasing. When a new value is larger than the top, pop and record the answer for the popped index.

5.8 Linked lists

```
/* Reverse #206 */
struct ListNode* reverseList(struct ListNode* head)
{
    struct ListNode *prev = NULL;
    while (head)
    {
        struct ListNode *next = head->next;
        head->next = prev;
        prev = head;
        head = next;
```

```
    }
    return prev;
}

/* Merge two sorted lists #21 with a dummy node */
struct ListNode* mergeTwoLists(struct ListNode* a, struct ListNode* b)
{
    struct ListNode dummy, *tail = &dummy;
    dummy.next = NULL;
    while (a && b)
    {
        if (a->val <= b->val) { tail->next = a; a = a->next; }
        else                  { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = a ? a : b;
    return dummy.next;
}

/* Cycle detection #141 (Floyd): slow moves 1, fast moves 2 */
bool hasCycle(struct ListNode *head)
{
    struct ListNode *slow = head, *fast = head;
    while (fast && fast->next)
    {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

Tip: Use a dummy node whenever the head can change (merge, remove node). Slow/fast pointers also find the middle (#876) and the Nth node from the end (#19, give the fast pointer an N-step head start).

5.9 Trees: DFS

```
/* Max Depth #104 */
int maxDepth(struct TreeNode* root)
{
    if (!root) return 0;
    int l = maxDepth(root->left), r = maxDepth(root->right);
    return 1 + (l > r ? l : r);
}

/* Invert #226 */
struct TreeNode* invertTree(struct TreeNode* root)
{
    if (!root) return NULL;
    struct TreeNode *tmp = root->left;
    root->left = invertTree(root->right);
    root->right = invertTree(tmp);
    return root;
}

/* Validate BST #98: pass the allowed range down (use long long bounds) */
static bool check(struct TreeNode *n, long long lo, long long hi)
{
    if (!n) return true;
    if (n->val <= lo || n->val >= hi) return false;
    return check(n->left, lo, n->val) && check(n->right, n->val, hi);
}
bool isValidBST(struct TreeNode* root) { return check(root, LLONG_MIN, LLONG_MAX); }
```

Recursion recipe for trees: (1) base case: `NULL`; (2) solve left and right; (3) combine their answers with the current node. Think of what each call must *return*.

5.10 BFS (level order)

```
/* Core loop; q is an array queue of pointers, N = max number of nodes */
struct TreeNode **q = malloc(N * sizeof(*q));
int head = 0, tail = 0;
q[tail++] = root;
while (head < tail)
{
    int level = tail - head;               /* nodes on this level */
    for (int i = 0; i < level; i++)
    {
        struct TreeNode *node = q[head++];
        /* visit node */
        if (node->left)  q[tail++] = node->left;
        if (node->right) q[tail++] = node->right;
    }
}
free(q);
```

5.11 Grid DFS (islands)

```
/* Number of Islands #200 */
static void dfs(char **g, int r, int c, int R, int C)
{
    if (r < 0 || c < 0 || r >= R || c >= C || g[r][c] != '1') return;
    g[r][c] = '0';                         /* mark visited */
    dfs(g, r + 1, c, R, C); dfs(g, r - 1, c, R, C);
    dfs(g, r, c + 1, R, C); dfs(g, r, c - 1, R, C);
}
int numIslands(char** grid, int gridSize, int* gridColSize)
{
    int R = gridSize, C = gridColSize[0], count = 0;
    for (int r = 0; r < R; r++)
        for (int c = 0; c < C; c++)
            if (grid[r][c] == '1') { count++; dfs(grid, r, c, R, C); }
    return count;
}
```

5.12 Backtracking

Template: choose, explore, un-choose. Subsets of an array with n <= 10 can be done with bitmasks, which avoids complicated memory handling in C:

```
/* Subsets #78 */
int** subsets(int* nums, int n, int* returnSize, int** returnColumnSizes)
{
    int total = 1 << n;
    int **res = malloc(total * sizeof(int *));
    *returnColumnSizes = malloc(total * sizeof(int));
    *returnSize = total;
    for (int mask = 0; mask < total; mask++)
    {
        int cnt = __builtin_popcount(mask);
        res[mask] = malloc((cnt ? cnt : 1) * sizeof(int));
        (*returnColumnSizes)[mask] = cnt;
        int k = 0;
        for (int i = 0; i < n; i++)
            if (mask & (1 << i)) res[mask][k++] = nums[i];
    }
    return res;
}

/* General backtracking skeleton */
void solve(state, choices)
{
    if (is_complete(state)) { record(state); return; }
    for (each choice in choices)
    {
```

```
        if (!valid(choice)) continue;
        apply(choice);              /* choose   */
        solve(state, choices);      /* explore  */
        undo(choice);               /* un-choose */
    }
}
```

5.13 Dynamic programming

Steps: (1) define what `dp[i]` means; (2) write the recurrence; (3) set base cases; (4) choose the fill order; (5) read the answer. Start recursive with memo if the recurrence is easier to see, then convert to a table.

```
/* Climbing Stairs #70: dp[i] = dp[i-1] + dp[i-2] (space O(1)) */
int climbStairs(int n)
{
    int a = 1, b = 1;
    for (int i = 2; i <= n; i++) { int t = a + b; a = b; b = t; }
    return b;
}

/* House Robber #198: dp[i] = max(dp[i-1], dp[i-2] + nums[i]) */
int rob(int* nums, int n)
{
    int prev2 = 0, prev1 = 0;
    for (int i = 0; i < n; i++)
    {
        int cur = prev1 > prev2 + nums[i] ? prev1 : prev2 + nums[i];
        prev2 = prev1; prev1 = cur;
    }
    return prev1;
}

/* Coin Change #322: dp[a] = min coins to make amount a */
int coinChange(int* coins, int n, int amount)
{
    int *dp = malloc((amount + 1) * sizeof(int));
    dp[0] = 0;
    for (int a = 1; a <= amount; a++)
    {
        dp[a] = amount + 1;                     /* "infinity" */
        for (int i = 0; i < n; i++)
            if (coins[i] <= a && dp[a - coins[i]] + 1 < dp[a])
                dp[a] = dp[a - coins[i]] + 1;
    }
    int res = dp[amount] > amount ? -1 : dp[amount];
    free(dp);
    return res;
}
```

5.14 Heap (priority queue)

C has no heap in the standard library, so write a small one. This is a min-heap of ints. For 'K largest', keep a min-heap of size K.

```
typedef struct { int *a; int size; } Heap;
static void swp(int *x, int *y) { int t = *x; *x = *y; *y = t; }

void push(Heap *h, int v)
{
    int i = h->size++;
    h->a[i] = v;
    while (i > 0 && h->a[(i - 1) / 2] > h->a[i])   /* sift up */
    {
        swp(&h->a[(i - 1) / 2], &h->a[i]);
        i = (i - 1) / 2;
    }
}
int pop(Heap *h)
```

```
{
    int top = h->a[0];
    h->a[0] = h->a[--h->size];
    int i = 0;
    for (;;)                                        /* sift down */
    {
        int l = 2 * i + 1, r = l + 1, m = i;
        if (l < h->size && h->a[l] < h->a[m]) m = l;
        if (r < h->size && h->a[r] < h->a[m]) m = r;
        if (m == i) break;
        swp(&h->a[m], &h->a[i]);
        i = m;
    }
    return top;
}
```

5.15 Bit manipulation

```
/* Single Number #136: pairs cancel out under XOR */
int singleNumber(int* nums, int n)
{
    int x = 0;
    for (int i = 0; i < n; i++) x ^= nums[i];
    return x;
}

/* Number of 1 Bits #191: x &= x - 1 clears the lowest set bit */
int hammingWeight(uint32_t n) { int c = 0; while (n) { n &= n - 1; c++; } return c; }

/* Counting Bits #338: bits[i] = bits[i >> 1] + (i & 1) */
/* Missing Number #268: XOR all indices 0..n with all values; or n(n+1)/2 - sum */
```

6. C-specific traps on LeetCode

Trap	What happens	Fix
Forgetting `*returnSize`	Wrong answer or crash	Set it on **every** return path, including empty results
Returning a local array	Dangling pointer, garbage values	malloc the result
Static / global variables	They persist between test cases and carry old data	Reset them at the start of the function, or avoid them
Integer overflow	Wrong sums / products	Use `long long`; compare to INT_MAX / LLONG_MAX
`abs(INT_MIN)`	Undefined behavior	Use long long first
`mid = (lo + hi) / 2`	Overflow for large indexes	`lo + (hi - lo) / 2`
Missing null terminator	Garbage after returned strings	Allocate len + 1 and set `'\0'`
`strlen` in a loop condition	Turns O(n) into O(n^2)	Compute it once before the loop
`size_t i` counting down	`i >= 0` is always true	Use int, or loop with `i-- > 0`
`sizeof(arr)` on a parameter	Gives the pointer size, not the array size	Use the size parameter you were given
Deep recursion (n = 10^5)	Stack overflow	Convert to an iterative solution with an explicit stack
Modifying input strings	Fails if the input is a literal or must be preserved	Copy first when unsure
Memory leaks in temporary buffers	Usually passes, but is bad habit	Free what you allocate, except the returned data

7. Roadmap: problems by topic

Difficulty: E = Easy, M = Medium. Do them in this order. Finish one topic before moving on, and re-solve any you needed help with after a few days.

Arrays and hashing

#	Problem	Diff	Key idea
217	Contains Duplicate	E	Hash set or sort
242	Valid Anagram	E	Counting array of 26
1	Two Sum	E	Hash map: complement lookup
49	Group Anagrams	M	Sort each word as the key, or count signature
347	Top K Frequent Elements	M	Count, then heap or bucket by frequency
238	Product of Array Except Self	M	Prefix and suffix products
128	Longest Consecutive Sequence	M	Hash set; only start at sequence beginnings
169	Majority Element	E	Boyer-Moore voting
283	Move Zeroes	E	Write pointer

Two pointers

#	Problem	Diff	Key idea
125	Valid Palindrome	E	Pointers from both ends
26	Remove Duplicates from Sorted Array	E	Read and write pointers
27	Remove Element	E	Read and write pointers
88	Merge Sorted Array	E	Merge from the back
11	Container With Most Water	M	Move the shorter side inward
15	3Sum	M	Sort, fix one, two pointers, skip duplicates

Sliding window

#	Problem	Diff	Key idea
121	Best Time to Buy and Sell Stock	E	Track min so far
3	Longest Substring Without Repeating Characters	M	Last-seen index per char
424	Longest Repeating Character Replacement	M	Window valid if len - maxCount <= k

Stack

#	Problem	Diff	Key idea
20	Valid Parentheses	E	Stack of openers
155	Min Stack	M	Store (value, min so far) pairs
150	Evaluate Reverse Polish Notation	M	Stack of operands
739	Daily Temperatures	M	Monotonic decreasing stack of indices

Binary search

#	Problem	Diff	Key idea
704	Binary Search	E	Template from section 5.5

#	Problem	Diff	Key idea
69	Sqrt(x)	E	Binary search on the answer, use long long
74	Search a 2D Matrix	M	Treat as one sorted array of m*n
153	Find Minimum in Rotated Sorted Array	M	Compare mid with right end
33	Search in Rotated Sorted Array	M	Find which half is sorted
875	Koko Eating Bananas	M	Binary search on speed

Linked lists

#	Problem	Diff	Key idea
206	Reverse Linked List	E	prev / curr / next
21	Merge Two Sorted Lists	E	Dummy node
141	Linked List Cycle	E	Slow and fast pointers
876	Middle of the Linked List	E	Slow and fast pointers
19	Remove Nth Node From End of List	M	Dummy node plus gap of N
2	Add Two Numbers	M	Digit by digit with carry
143	Reorder List	M	Find middle, reverse second half, merge

Trees

#	Problem	Diff	Key idea
104	Maximum Depth of Binary Tree	E	1 + max(left, right)
226	Invert Binary Tree	E	Swap children recursively
100	Same Tree	E	Compare node and both subtrees
110	Balanced Binary Tree	E	Return height, or -1 if unbalanced
543	Diameter of Binary Tree	E	Height plus a global best
102	Binary Tree Level Order Traversal	M	BFS with level size
98	Validate Binary Search Tree	M	Pass min and max bounds down
235	Lowest Common Ancestor of a BST	M	Walk down using value comparisons

Backtracking

#	Problem	Diff	Key idea
78	Subsets	M	Bitmask or choose/skip
46	Permutations	M	Swap-based or used[] array
39	Combination Sum	M	Reuse allowed: recurse at same index
79	Word Search	M	Grid DFS, mark and unmark visited

Heap

#	Problem	Diff	Key idea
703	Kth Largest Element in a Stream	E	Min-heap of size K
215	Kth Largest Element in an Array	M	Min-heap of size K, or quickselect

Graphs

#	Problem	Diff	Key idea
200	Number of Islands	M	Grid DFS or BFS
695	Max Area of Island	M	DFS returning area
133	Clone Graph	M	DFS with an old-to-new map
207	Course Schedule	M	Cycle detection / topological sort

Dynamic programming

#	Problem	Diff	Key idea
70	Climbing Stairs	E	Fibonacci
198	House Robber	M	Take or skip
62	Unique Paths	M	dp[r][c] = dp[r-1][c] + dp[r][c-1]
322	Coin Change	M	Unbounded knapsack
300	Longest Increasing Subsequence	M	O(n^2) dp first, then O(n log n)
139	Word Break	M	dp[i] = some word ends at i and dp[j] is true
1143	Longest Common Subsequence	M	2D dp over both strings
53	Maximum Subarray	M	Kadane

Bit manipulation and math

#	Problem	Diff	Key idea
136	Single Number	E	XOR everything
191	Number of 1 Bits	E	n &= n - 1
338	Counting Bits	E	bits[i] = bits[i/2] + (i%2)
190	Reverse Bits	E	Shift out and in, 32 times
268	Missing Number	E	XOR or sum formula
9	Palindrome Number	E	Reverse half the number
13	Roman to Integer	E	Subtract if next symbol is larger
66	Plus One	E	Carry from the end
67	Add Binary	E	Carry from the end of both strings

Arrays and matrices (extra)

#	Problem	Diff	Key idea
14	Longest Common Prefix	E	Compare column by column
28	Find the Index of the First Occurrence in a String	E	Naive scan
56	Merge Intervals	M	Sort by start, then merge
189	Rotate Array	M	Triple reverse
48	Rotate Image	M	Transpose, then reverse rows
54	Spiral Matrix	M	Four shrinking boundaries
73	Set Matrix Zeroes	M	Use first row/col as markers

8. An 8-week study plan

About 1 hour a day, 5-6 days a week. Aim for 3-5 problems per week per topic early on, quality over quantity.

Week	Focus	Goal
1	C fundamentals, arrays, hashing, counting	Comfortable with returnSize, malloc, and building a hash table
2	Two pointers, sliding window, prefix sums	Recognize these from the problem statement
3	Stack, binary search	Write binary search templates from memory
4	Linked lists	Reverse, merge, cycle, and dummy-node patterns without bugs
5	Trees (DFS and BFS)	Think recursively: base case plus combine
6	Backtracking and heap	Choose / explore / un-choose; write a heap
7	Graphs and dynamic programming	Grid DFS/BFS; define dp state before coding
8	Mixed review, timed practice	Solve mediums in 30 minutes; redo past misses

9. Habits that make you improve

- **Keep a mistake log.** For each problem you miss, write the pattern, the trick, and why you missed it.
- **Re-solve after 3 days and again after 2 weeks.** Spaced repetition is what moves a pattern into long-term memory.
- **Say your approach before coding.** This is exactly what interviews test.
- **Always state time and space complexity** of your solution.
- **Do timed sessions** (30-45 minutes per problem) once you are past week 4.
- **Compare solutions.** After solving, read a top solution and note what was cleaner or faster.
- **Do not memorize code, memorize patterns.** If you know the why, the code follows.
- **Be consistent.** One problem a day beats ten on Sunday.

Quick checklist before you submit

- Did I handle the empty / one-element / all-equal cases?
- Did I set `*returnSize` (and returnColumnSizes if needed) on every return path?
- Is every returned array or string malloc'd and null-terminated?
- Could any sum or product overflow an int?
- Do my loop bounds avoid reading outside the array?
- Did I free temporary memory I allocated?
- Do global or static variables carry state between test cases?